

# Deploying to Oracle Cloud (free forever)

---

Gets the portal running permanently on a machine that is always on, so the site stays up and the collector keeps working when your laptop is off.

Oracle's **Always Free** tier is free indefinitely, not a trial. The ARM machine it offers — 4 cores and 24 GB of memory — is far more than this needs.

---

## Part 1: the account

---

**A card is required to verify identity. You are not charged.** Oracle places a small authorisation hold, usually around ₹100, which is released. The account stays on Always Free unless you deliberately upgrade it.

1. Go to [cloud.oracle.com](https://cloud.oracle.com) and choose **Start for free**.
  2. **Choose your home region carefully — it cannot be changed later.** Pick **India South (Hyderabad)** or **India West (Mumbai)**. Keeping the data in India removes a category of questions under the DPDP Act.
  3. Complete the card verification.
  4. Wait for the "your account is ready" email. Usually minutes, occasionally a few hours.
- 

## Part 2: the machine

---

In the console: **Menu → Compute → Instances → Create instance.**

| Setting     | Value                        | Why                                                    |
|-------------|------------------------------|--------------------------------------------------------|
| Image       | <b>Ubuntu 24.04</b>          | Not Oracle Linux; the instructions below assume Ubuntu |
| Shape       | <b>VM.Standard.A1.Flex</b>   | The ARM one. Set 4 OCPU and 24 GB                      |
| Boot volume | 50 GB                        | Within the free allowance                              |
| SSH keys    | <b>Paste your public key</b> | How you log in                                         |

Check it says **"Always Free-eligible"** before you create it.

### If it says "Out of capacity"

Common, and not your mistake — free ARM capacity in Indian regions is often exhausted. Options, in order of preference:

1. **Try again later.** Capacity frees up constantly; early morning tends to work. Many people get in within a day.
2. **Try the other availability domain** in the same region.
3. **Use VM.Standard.E2.1.Micro** instead. Also Always Free, available immediately, but 1 GB of memory. This project will run on it, though a crawl will be slow and you should reduce `CRAWL_BATCH_SIZE` to about 300.

## Your SSH key

If you do not have one:

```
ssh-keygen -t ed25519 -C "auction-portal"  
# then paste the contents of C:\Users\<you>\.ssh\id_ed25519.pub
```

## Part 3: open the firewall

**Oracle blocks everything except SSH, in two separate places.** Miss either and the site appears dead. This is the single most common reason a first Oracle deployment "doesn't work".

### 3a. The cloud firewall

**Networking** → **Virtual Cloud Networks** → your VCN → **Security Lists** → **Default Security List** → **Add Ingress Rules**:

| Source CIDR | Protocol | Destination port |
|-------------|----------|------------------|
| 0.0.0.0/0   | TCP      | 80               |
| 0.0.0.0/0   | TCP      | 443              |

### 3b. The firewall on the machine itself

Oracle's Ubuntu images ship with iptables rules that block these ports even after you open them above. SSH in and run:

```
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 80 -j ACCEPT  
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 443 -j ACCEPT  
sudo netfilter-persistent save
```

## Part 4: install and deploy

SSH in as `ubuntu@<your-public-ip>`, then:

```
# Docker  
curl -fsSL https://get.docker.com | sudo sh  
sudo usermod -aG docker ubuntu  
newgrp docker  
  
# The project  
git clone https://github.com/YOUR-USERNAME/YOUR-REPO.git auction-portal  
cd auction-portal
```

Create `.env` on the server — **never commit this file**:

```
cat > .env <<'EOF'
ENVIRONMENT=production
POSTGRES_PASSWORD=<a long random password>
CONTACT_EMAIL=you@example.com

# A hostname Let's Encrypt can verify. nip.io resolves any IP embedded
# in it, so this gives free HTTPS with no domain to buy. Replace the
# digits with your actual public IP.
SITE_ADDRESS=203.0.113.10.nip.io
CONTACT_URL=https://203.0.113.10.nip.io/bot
EOF
```

Then:

```
docker compose up -d --build
docker compose run --rm web alembic upgrade head
docker compose logs -f
```

The first build takes five to ten minutes on ARM.

Your site is at <https://<your-ip>.nip.io> — with a real certificate, because `nip.io` resolves to the IP and Let's Encrypt can verify it.

---

## Part 5: bring your existing listings across

---

Optional. Skipping it just means the collector rebuilds coverage over the following fortnight.

On your laptop, with the local database running:

```
.\pgsql\pgsql\bin\pg_dump.exe -U postgres -p 5433 -h localhost `
-d auction_portal -Fc -f listings.dump
scp listings.dump ubuntu@<your-ip>:~/auction-portal/
```

On the server:

```
docker compose cp listings.dump postgres:/tmp/
docker compose exec postgres pg_restore -U auction -d auction_portal \
--clean --if-exists /tmp/listings.dump
```

The dump is a few megabytes. The 363 MB document archive is deliberately left behind: it is re-fetchable, and nothing depends on it except re-parsing history.

---

## Part 6: check it

---

From anywhere:

```
.\.venv\Scripts\python.exe scripts\smoke_check.py https://<your-ip>.nip.io
.\.venv\Scripts\python.exe scripts\verify_manual.py https://<your-ip>.nip.io
```

On the server:

```
docker compose ps # all four up
docker compose exec web python -m auction_portal health
docker compose logs -f scheduler
```

---

## Afterwards

**Turn off the laptop autostart**, or you will have two collectors crawling BAANKNET at once and double the request rate:

```
powershell -ExecutionPolicy Bypass -File scripts\install_autostart.ps1 -Disable
```

**Updating** is `git pull && docker compose up -d --build`. The database is untouched by deployments.

**A real domain** later is a one-line change: point it at the IP, set `SITE_ADDRESS` to it, and restart Caddy. The certificate is handled automatically.

---

## Things that will catch you out

**"Out of capacity" on ARM.** Expected. Retry, or start on `E2.1.Micro`.

**Site unreachable after deploying.** Almost always the firewall, and almost always the second one — the iptables rules in Part 3b, not the cloud rule in 3a.

**The home region is permanent.** Chosen at signup, never changeable.

**Always Free has limits:** 4 ARM cores and 24 GB total across all your instances, and 200 GB of block storage. One machine at 4/24 uses your whole ARM allowance, which is fine.

**Do not upgrade the account** to pay-as-you-go unless you mean to. Always Free resources stay free either way, but it removes the guard rail that stops you accidentally creating something billable.